

# Unidad 10: Playground Intermedio

## Parte I

### Profundizando en MTV

El patrón de arquitectura MTV (Model-Template-View) es una variante del patrón MVC (Model-View-Controller) utilizado en Django, un popular framework de desarrollo web en Python. Este patrón ayuda a organizar el código de manera eficiente y facilita la separación de preocupaciones dentro de una aplicación web.

En esta sección, explicaremos cómo crear un proyecto y su respectiva aplicación en Django, profundizando en cada capa del patrón MTV utilizando como ejemplo la AppCoder, que incluye modelos para estudiantes, profesores, entregables y cursos.

---

## Creación de un Proyecto y Aplicación en Django

### Paso 1: Configuración del Proyecto

Primero, necesitamos crear un proyecto Django. Para ello, utilizamos el comando ``django-admin startproject`` seguido del nombre del proyecto. Supongamos que nuestro proyecto se llama ``MiProyecto``.

```
```bash
django-admin startproject MiProyecto .
cd MiProyecto
```
```

### Paso 2: Creación de la Aplicación

Dentro del proyecto, creamos una aplicación llamada ``AppCoder``.

```
```bash
python manage.py startapp AppCoder
```
```

---

# Modelo (Model)

El modelo es la capa que interactúa con la base de datos. Define la estructura de los datos y cómo se almacenan. En la aplicación `AppCoder`, creamos los modelos para estudiantes, profesores, entregables y cursos.

## Definición de Modelos

En el archivo `models.py` de `AppCoder`, definimos nuestros modelos:

```
```python

from django.db import models

class Estudiante(models.Model):
    nombre = models.CharField(max_length=100)
    apellido = models.CharField(max_length=100)
    email = models.EmailField()

    def __str__(self):
        return f"{self.nombre} {self.apellido}"

class Profesor(models.Model):
    nombre = models.CharField(max_length=100)
    apellido = models.CharField(max_length=100)
    email = models.EmailField()
    profesion = models.CharField(max_length=100)

    def __str__(self):
        return f"{self.nombre} {self.apellido} - {self.profesion}"

class Curso(models.Model):
    nombre = models.CharField(max_length=100)
    camada = models.IntegerField()

    def __str__(self):
        return self.nombre

class Entregable(models.Model):
    nombre = models.CharField(max_length=100)
    fecha_entrega = models.DateField()
    entregado = models.BooleanField()

    def __str__(self):
        return self.nombre
```
```

Estos modelos representan las entidades principales de nuestra aplicación y se traducen en tablas de base de datos.

---

## Agregar nuestra app al proyecto

Para que el proyecto de Django reconozca nuestra App debemos agregarla en los setting del mismo, dentro de `**settings.py**` agregamos

```
```bash

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'AppCoder', # El nombre de nuestra app
]
```
```

---

## Migración de Modelos

Para crear las tablas en la base de datos, ejecutamos las migraciones:

```
```bash

python manage.py makemigrations
python manage.py migrate
```
```

---

## Vista (View)

La vista maneja la lógica de negocio y la interacción del usuario. Se encarga de procesar las solicitudes y devolver respuestas. En `AppCoder`, creamos vistas para manejar las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) para nuestros modelos.

## Definición de Vistas

En el archivo `views.py` de `AppCoder`, definimos nuestras vistas:

```
```python

from django.shortcuts import render, get_object_or_404
from .models import Estudiante, Profesor, Curso, Entregable

def lista_estudiantes(request):
    estudiantes = Estudiante.objects.all()
    return render(request, 'AppCoder/estudiantes_list.html', {'estudiantes': estudiantes})

def detalle_estudiante(request, pk):
    estudiante = get_object_or_404(Estudiante, pk=pk)
    return render(request, 'AppCoder/estudiante_detail.html', {'estudiante': estudiante})
```
```

Estas vistas obtienen datos de los modelos y los pasan a las plantillas para su presentación.

---

## Plantilla (Template)

Las plantillas definen cómo se presentan los datos al usuario. Utilizan el lenguaje de plantillas de Django para generar HTML dinámico.

### Definición de Plantillas

Creamos las plantillas en el directorio `templates/AppCoder/`.

**Plantilla `estudiantes\_list.html`:**

```
```html

<!DOCTYPE html>
<html>
<head>
    <title>Lista de Estudiantes</title>
</head>
<body>
    <h1>Lista de Estudiantes</h1>
    <ul>
        {% for estudiante in estudiantes %}
            <li>{{ estudiante.nombre }} {{ estudiante.apellido }}</li>
        {% endfor %}
    </ul>
</body>
```
```

```
</html>
'''
```

**Plantilla `estudiante\_detail.html`:**

```
'''html

<!DOCTYPE html>
<html>
<head>
    <title>Detalle del Estudiante</title>
</head>
<body>
    <h1>{{ estudiante.nombre }} {{ estudiante.apellido }}</h1>
    <p>Email: {{ estudiante.email }}</p>
</body>
</html>
'''
```

---

## Configuración de URLs

Finalmente, configuramos las URLs para que las vistas sean accesibles. Editamos el archivo `urls.py` de `AppCoder` y el archivo principal de URLs del proyecto.

**`AppCoder/urls.py`:**

```
'''python

from django.urls import path
from .views import lista_estudiantes, detalle_estudiante

urlpatterns = [
    path('estudiantes/', lista_estudiantes, name='lista_estudiantes'),
    path('estudiantes/<int:pk>/', detalle_estudiante, name='detalle_estudiante'),
]
'''
```

**`MiProyecto/urls.py`:**

```
'''python

from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
```

```
path('appcoder/', include('AppCoder.urls')),  
]  
...
```

---

## Conclusión

El patrón MTV en Django ayuda a organizar el código de manera eficiente. Hemos creado un proyecto y una aplicación `AppCoder` que incluye modelos, vistas y plantillas para manejar estudiantes, profesores, entregables y cursos. Con esta estructura, es más fácil gestionar y mantener la aplicación a medida que crece en complejidad.

---

## Control de Versiones

El control de versiones es una práctica esencial en el desarrollo de software que permite gestionar los cambios realizados en el código a lo largo del tiempo. Utilizar un sistema de control de versiones evita la pérdida de trabajo, facilita la colaboración entre desarrolladores y permite volver a puntos seguros en el desarrollo del proyecto.

### Importancia del Control de Versiones

El control de versiones ofrece varios beneficios cruciales para el desarrollo de software:

- **Seguridad:** Permite recuperar versiones anteriores del proyecto, evitando la pérdida de trabajo ante errores o problemas.
  - **Colaboración:** Facilita el trabajo en equipo, permitiendo que múltiples desarrolladores trabajen en el mismo proyecto sin interferencias.
  - **Historial:** Mantiene un registro detallado de todos los cambios realizados, quién los hizo y cuándo se hicieron.
  - **Ramas:** Permite desarrollar nuevas funcionalidades en paralelo sin afectar la versión principal del proyecto.
- 

## Conceptos Básicos de Git y GitHub

### Repositorio

Un repositorio es una estructura de almacenamiento donde se guarda el historial de versiones de un proyecto. Puede estar en tu máquina local (repositorio local) o en un servidor remoto como GitHub (repositorio remoto).

## Commits

Un commit es una instantánea del proyecto en un momento específico. Incluye un mensaje descriptivo que ayuda a entender los cambios realizados. Hacer commits regularmente permite crear un historial detallado del proyecto.

**Ejemplo de comando para hacer un commit:**

```
```bash
git commit -m "Descripción de los cambios realizados"
```
```

---

## Ramas (Branches)

Las ramas permiten desarrollar funcionalidades nuevas de manera aislada del código principal (rama principal o master). Una vez que la funcionalidad está lista y probada, se puede fusionar (merge) con la rama principal.

**Ejemplo de comando para crear una nueva rama:**

```
```bash
git branch nombre_de_la_rama
```
```

## Fusión (Merge)

La fusión es el proceso de incorporar cambios de una rama en otra. Esto es comúnmente utilizado para integrar nuevas funcionalidades en la rama principal del proyecto.

**Ejemplo de comando para fusionar una rama:**

```
```bash
git merge nombre_de_la_rama
```
```

---

## Usando Git y GitHub

### Configuración Inicial

Primero, debes instalar Git y configurarlo con tu nombre de usuario y correo electrónico:

```
```bash
```

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tuemail@example.com"  
```
```

## Creación de un Repositorio

Para iniciar un nuevo repositorio, navega a la carpeta de tu proyecto y ejecuta:

```
```bash
```

```
git init  
```
```

---

## Añadir Archivos al Repositorio

Añade archivos al área de preparación (staging area) para incluirlos en el próximo commit:

```
```bash
```

```
git add nombre_del_archivo  
```
```

Para añadir todos los archivos modificados:

```
```bash
```

```
git add .  
```
```

## Hacer un Commit

Guarda los cambios en el historial del proyecto:

```
```bash
```

```
git commit -m "Descripción de los cambios"  
```
```

---



# Conectar con GitHub

Para subir tu repositorio a GitHub, primero crea un repositorio en GitHub. Luego, conecta tu repositorio local al remoto:

```
``bash

git remote add origin https://github.com/tuusuario/tu-repositorio.git
``
```

## Subir Cambios a GitHub

Para subir tus commits al repositorio remoto en GitHub:

```
``bash

git push origin master
``
```

## Clonar un Repositorio

Si deseas trabajar en un proyecto existente en GitHub, puedes clonarlo:

```
``bash

git clone https://github.com/tuusuario/tu-repositorio.git
``
```

---

# Flujo de Trabajo Básico

### 1. Crear una nueva rama para una nueva funcionalidad:

```
``bash

git checkout -b nueva_funcionalidad
``
```

### 2. Trabajar en la nueva funcionalidad y hacer commits regularmente:

```
``bash

git add .
```

```
git commit -m "Implementa nueva funcionalidad"
'''
```

### 3. Cambiar a la rama principal y fusionar los cambios:

```
```bash

git checkout master
git merge nueva_funcionalidad
'''
```

### 4. Subir los cambios a GitHub:

```
```bash

git push origin master
'''
```

---

## Conclusión

El control de versiones con Git y GitHub es una práctica esencial para el desarrollo de software moderno. Facilita la colaboración, protege contra la pérdida de datos y mantiene un historial detallado de los cambios. Aprender a utilizar estas herramientas te permitirá gestionar tus proyectos de manera eficiente y profesional.

---

## URLs Avanzada

En Django, la organización y gestión de URLs es crucial para la estructura y navegación de un proyecto web. Las URLs actúan como puntos de entrada para acceder a las diferentes vistas de tu aplicación. Tener un archivo `urls.py` bien organizado es fundamental para manejar múltiples aplicaciones (apps) dentro de un proyecto Django. Este enfoque no solo mejora la mantenibilidad del código, sino que también facilita la ampliación del proyecto a medida que crece en complejidad.

### Importancia de un Archivo `urls.py` Bien Organizado

El archivo `urls.py` es donde defines todas las rutas URL que Django debe reconocer y mapear a las vistas correspondientes. Una buena organización de este archivo permite:

- **Facilidad de Mantenimiento:** Es más sencillo localizar y actualizar las rutas URL.
- **Escalabilidad:** Facilita la adición de nuevas aplicaciones y funcionalidades.

- **Claridad:** Ayuda a tener una visión clara de la estructura y flujo de la aplicación.

En un proyecto Django, el archivo `urls.py` principal se encuentra en el directorio del proyecto. Sin embargo, para proyectos con múltiples aplicaciones, es recomendable tener un archivo `urls.py` separado dentro de cada app. Esto ayuda a gestionar de manera efectiva las URLs específicas de cada aplicación.

---

## Organización de URLs

Cuando se trabaja con múltiples aplicaciones en Django, es esencial organizar las URLs de una manera que sea modular y fácil de gestionar. Una buena práctica es crear un archivo `urls.py` dentro de cada aplicación. Luego, el archivo `urls.py` principal del proyecto puede incluir estas rutas.

## Configuración Básica de URLs en el Proyecto

Supongamos que tenemos un proyecto llamado `MiProyecto` con dos aplicaciones: `AppBlog` y `AppTienda`.

### Paso 1: Archivo `urls.py` del Proyecto

En el archivo `urls.py` del proyecto (`MiProyecto/urls.py`), incluimos las rutas de cada aplicación.

**MiProyecto/urls.py:**

```
```python

from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('blog/', include('AppBlog.urls')),
    path('tienda/', include('AppTienda.urls')),
]
```
```

### Paso 2: Archivo `urls.py` de Cada Aplicación

Cada aplicación tendrá su propio archivo `urls.py`, donde se definirán las rutas específicas de esa aplicación.

**AppBlog/urls.py:**

```
```python
```

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.index, name='index'),
    path('post/<int:id>/', views.detalle_post, name='detalle_post'),
    path('categoria/<str:categoria>/', views.categoria, name='categoria'),
]
'''
```

#### **AppTienda/urls.py:**

```
'''python

from django.urls import path
from . import views

urlpatterns = [
    # Iremos agregando los endpoint de esta app aquí
]
'''
```

---

## **Ventajas de una Buena Organización de URLs**

### **1. Modularidad**

Cada aplicación gestiona sus propias URLs, lo que facilita la organización del código y la identificación de rutas específicas.

### **2. Aislamiento**

Permite a los desarrolladores trabajar en diferentes aplicaciones sin interferir con las rutas de otras partes del proyecto.

### **3. Reutilización**

Las aplicaciones pueden ser reutilizadas en diferentes proyectos con una configuración mínima.

---

## **Ejemplo Completo**

Supongamos que quieres agregar una nueva funcionalidad en `AppBlog` para listar todos los posts de una categoría específica.

## Paso 1: Definir la Vista

**AppBlog/views.py:**

```
```python

from django.shortcuts import render

def categoria(request, categoria):
    # implementar función obtener_posts_por_categoria
    posts = obtener_posts_por_categoria(categoria)
    return render(request, 'AppBlog/categoria.html', {'posts': posts})
...```
```

## Paso 2: Actualizar `urls.py` de la Aplicación

**AppBlog/urls.py:**

```
```python

from django.urls import path
from . import views

urlpatterns = [
    path("", views.index, name='index'),
    path('post/<int:id>/', views.detalle_post, name='detalle_post'),
    path('categoria/<str:categoria>', views.categoria, name='categoria'), # agregamos el
nuevo endpoint
]
...```
```

## Paso 3: Incluir las URLs de la Aplicación en el Proyecto Principal

**MiProyecto/urls.py:**

```
```python

from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('blog/', include('AppBlog.urls')),
    path('tienda/', include('AppTienda.urls')),
]
...```
```

'''

---

## Conclusión

Organizar las URLs de un proyecto Django de manera modular y clara es esencial para mantener la eficiencia y escalabilidad del desarrollo. Al crear un archivo `urls.py` dentro de cada aplicación y gestionarlo desde el archivo principal `urls.py` del proyecto, se logra una estructura bien definida que facilita el trabajo en equipo y la expansión del proyecto. Con estas prácticas, estarás mejor preparado para manejar proyectos Django complejos y en crecimiento.

---

## Templates

En Django, los templates son archivos HTML que definen la estructura y presentación de las páginas web que se envían al navegador del usuario. Los templates permiten separar la lógica del negocio (controladores) de la presentación (vistas), facilitando la organización y mantenimiento del código.

## Pasos para Crear y Organizar Templates en Django

### Paso 1: Configuración del Directorio de Templates

Django busca los templates en los directorios especificados en la configuración del proyecto. Es recomendable tener una estructura clara y organizada para los templates.

- 1. Crear el directorio de templates:** Dentro de la aplicación, crea un directorio llamado `templates`.
- 2. Crear subdirectorios para cada aplicación:** Dentro del directorio `templates`, crea subdirectorios específicos para cada aplicación para mantener los templates organizados.

### Ejemplo de estructura:

'''

```
MiProyecto/  
  AppBlog/  
    templates/  
      AppBlog/  
        index.html  
        detalle_post.html  
  AppTienda/  
    templates/
```

```
AppTienda/
  index.html
  detalle_producto.html
...

```

## Paso 2: Configuración de la Ruta de Templates en `settings.py`

En el archivo `settings.py` del proyecto, añade el directorio de templates a la configuración de `TEMPLATES`.

**`settings.py`:**

```
```python

import os

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')], # agregamos la ubicación de los
templates
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]
...

```

## Paso 3: Creación de Modelo

**Creamos el modelo para guardar los Post:** Dentro de el archivo [models.py](http://models.py) de nuestra aplicación creamos el modelo para guardar nuestros Post en la base de datos.

```
```python
from django.db import models

class Post(models.Model):

```

```

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    fecha_publicacion = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.titulo # Devuelve el título del post como su representación en cadena
...

```

Desde la consola corremos las migraciones

```

```bash
python manage.py makemigrations
python manage.py migrate
```

```

## Paso 4: Creación de Templates

**Crear un template base:** Es una buena práctica tener un template base que contenga la estructura común de la página, como el encabezado, pie de página y otras partes repetitivas.

**`base.html`:**

```

```html

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Mi Sitio{% endblock %}</title>
</head>
<body>
    <header>
        <h1>Mi Sitio Web</h1>
        <nav>
            <ul>
                <li><a href="#">Inicio</a></li>
                <li><a href="{% url 'blog:index' %}">Blog</a></li>
                <li><a href="#">Tienda</a></li>
            </ul>
        </nav>
    </header>
    <main>
        {% block content %}
        {% endblock %}
    </main>
    <footer>

```



```
<p>&copy; 2024 Mi Sitio Web</p>
</footer>
</body>
</html>
...

```

---

**Crear templates específicos para cada vista:** Utiliza el template base para extender los templates específicos de cada vista.

**`index.html` de `AppBlog`:**

```
```html

{% extends "base.html" %}

{% block title %}Inicio - Blog{% endblock %}

{% block content %}
<h2>Bienvenido al Blog</h2>
<ul>
    {% for post in posts %}
    <li><a href="{% url 'detalle_post' post.id %}">{{ post.titulo }}</a></li>
    {% endfor %}
</ul>
{% endblock %}
...

```

---

## Paso 5: Renderizar Templates desde las Vistas

Las vistas en Django deben renderizar los templates y pasar los datos necesarios para su presentación.

**`views.py` de `AppBlog`:**

```
```python

from django.shortcuts import render
from .models import Post

def index(request):
    posts = Post.objects.all()
    return render(request, 'AppBlog/index.html', {'posts': posts})

def detalle_post(request, id):
    post = Post.objects.get(pk=id)
    return render(request, 'AppBlog/detalle_post.html', {'post': post})

```

---

## Importancia de una Estructura Clara

Tener una estructura clara y bien organizada para los templates es fundamental para el mantenimiento y escalabilidad del proyecto. Permite:

- **Facilidad de Mantenimiento:** Encontrar y actualizar templates específicos sin dificultad.
  - **Reutilización:** Utilizar templates base y bloques para evitar duplicación de código.
  - **Colaboración:** Facilitar la colaboración entre múltiples desarrolladores al mantener una organización coherente.
- 

## Resumen

1. **Configurar el directorio de templates:** Crear y configurar directorios específicos para cada aplicación.
2. **Configurar las rutas en `settings.py`:** Asegurarse de que Django busque templates en los directorios adecuados.
3. **Crear templates base y específicos:** Diseñar templates base para la estructura común y extenderlos en los templates específicos de cada vista.
4. **Renderizar templates desde las vistas:** Utilizar la función `render()` en las vistas para pasar datos y renderizar templates.

Con estos pasos, estarás bien preparado para manejar la creación y organización de templates en Django de manera eficiente y profesional.

---

## Creación de Modelos en Django

En Django, un modelo es una representación de una tabla en la base de datos. Cada modelo es una clase de Python que subclasea `django.db.models.Model`. Los modelos definen la estructura de los datos que se almacenan y cómo se interactúa con ellos a través de métodos de consulta. Los modelos son una parte fundamental en la estructura de un proyecto Django, ya que permiten la abstracción y manipulación de datos de manera eficiente y coherente.

### ¿Qué es un Modelo en Django?

Un modelo en Django define los campos y comportamientos de los datos que deseas almacenar. Los modelos proporcionan una API de alto nivel para crear, leer, actualizar y eliminar registros en la base de datos sin necesidad de escribir SQL directamente.

---

# Importancia de los Modelos en Django

- **Abstracción de la Base de Datos:** Permiten trabajar con datos sin preocuparse por los detalles específicos del SQL.
  - **Consistencia:** Facilitan la validación y la consistencia de los datos.
  - **Flexibilidad:** Proporcionan una manera sencilla de migrar cambios en la estructura de datos a lo largo del desarrollo del proyecto.
- 

## Proceso para Crear un Modelo en Django

A continuación, se detalla el proceso para crear un modelo en la aplicación `AppCoder` de un proyecto Django.

### Paso 1: Definir el Modelo

Para definir un modelo, necesitas editar el archivo `models.py` de tu aplicación. Vamos a crear modelos para `Estudiante`, `Profesor`, `Curso` y `Entregable` en `AppCoder`.

`AppCoder/models.py`:

### Paso 2: Realizar Migraciones

Django utiliza un sistema de migraciones para aplicar los cambios en la estructura de la base de datos. Después de definir los modelos, necesitas crear y aplicar las migraciones.

1. **Crear las migraciones:** Genera los archivos de migración que describen los cambios en los modelos.

```
```bash  
  
python manage.py makemigrations AppCoder  
```
```

2. **Aplicar las migraciones:** Aplica los cambios en la base de datos.

```
```bash  
  
python manage.py migrate  
```
```

## Paso 3: Registrar los Modelos en el Administrador de Django

Para poder gestionar los modelos desde el administrador de Django, debes registrarlos en el archivo `admin.py` de tu aplicación.

**`AppCoder/admin.py`:**

```
```python

from django.contrib import admin
from .models import Estudiante, Profesor, Curso, Entregable

admin.site.register(Estudiante)
admin.site.register(Profesor)
admin.site.register(Curso)
admin.site.register(Entregable)
```
```

## Paso 4: Crear y Probar el Superusuario

Para acceder al panel de administración, necesitas crear un superusuario (administrador).

### 1. Crear superusuario:

```
```bash

python manage.py createsuperuser
```
```

### 2. Iniciar el servidor de desarrollo:

```
```bash

python manage.py runserver
```
```

**3. Acceder al panel de administración:** Visita `http://127.0.0.1:8000/admin/` en tu navegador e inicia sesión con las credenciales del superusuario que creaste.

---

## Ejemplo Completo

A continuación se muestra un ejemplo completo del proceso para crear y gestionar modelos en la aplicación `AppCoder`.

## 1. Definir modelos en `models.py`:

```
```python

from django.db import models

class Estudiante(models.Model):
    nombre = models.CharField(max_length=100)
    apellido = models.CharField(max_length=100)
    email = models.EmailField()

    def __str__(self):
        return f'{self.nombre} {self.apellido}'

class Profesor(models.Model):
    nombre = models.CharField(max_length=100)
    apellido = models.CharField(max_length=100)
    email = models.EmailField()
    profesion = models.CharField(max_length=100)

    def __str__(self):
        return f'{self.nombre} {self.apellido} - {self.profesion}'

class Curso(models.Model):
    nombre = models.CharField(max_length=100)
    camada = models.IntegerField()

    def __str__(self):
        return self.nombre

class Entregable(models.Model):
    nombre = models.CharField(max_length=100)
    fecha_entrega = models.DateField()
    entregado = models.BooleanField()

    def __str__(self):
        return self.nombre
```
```

## 2. Crear y aplicar migraciones:

```
```bash

python manage.py makemigrations AppCoder
python manage.py migrate
```
```

### 3. Registrar modelos en `admin.py`:

```
```python

from django.contrib import admin
from .models import Estudiante, Profesor, Curso, Entregable

admin.site.register(Estudiante)
admin.site.register(Profesor)
admin.site.register(Curso)
admin.site.register(Entregable)

```
```

### 4. Crear superusuario y ejecutar el servidor:

```
```bash

python manage.py createsuperuser
python manage.py runserver

```
```

Accede al panel de administración de Django para gestionar los datos de `Estudiante`, `Profesor`, `Curso` y `Entregable`.

---

## Conclusión

La creación de modelos en Django es un paso esencial para definir la estructura de los datos en tu proyecto. Siguiendo estos pasos, puedes crear modelos que representen tus datos, aplicar cambios en la base de datos mediante migraciones y gestionar estos datos de manera eficiente a través del panel de administración de Django. Esta organización facilita el desarrollo y mantenimiento de aplicaciones web robustas y escalables.

---

## Bases de datos

Las migraciones en Django son un mecanismo que permite aplicar cambios en la estructura de la base de datos sin perder los datos existentes. Esto es esencial para el desarrollo ágil, ya que facilita la evolución del esquema de la base de datos junto con el código del proyecto.

# Proceso de Migraciones en Django

El proceso de migraciones en Django se realiza en dos pasos principales: crear las migraciones y aplicar las migraciones.

## Paso 1: Crear las Migraciones

El comando ``makemigrations`` se utiliza para crear nuevas migraciones basadas en los cambios realizados en los modelos. Este comando analiza los modelos y genera un archivo de migración que describe los cambios que deben aplicarse a la base de datos.

### Ejemplo:

```
```bash  
  
python manage.py makemigrations  
```
```

Este comando genera archivos de migración en el directorio ``migrations`` de cada aplicación. Estos archivos contienen instrucciones sobre cómo aplicar los cambios en la estructura de la base de datos.

## Paso 2: Aplicar las Migraciones

El comando ``migrate`` se utiliza para aplicar las migraciones pendientes a la base de datos. Este comando lee los archivos de migración y ejecuta las instrucciones para actualizar la base de datos según sea necesario.

### Ejemplo:

```
```bash  
  
python manage.py migrate  
```
```

Este comando asegura que la base de datos esté sincronizada con los modelos definidos en el código. También puede manejar migraciones más complejas, como cambios en los tipos de datos y relaciones entre tablas.

---

## Comandos Clave

- ``makemigrations``: Crea archivos de migración basados en los cambios en los modelos.

```
```bash
```

```
python manage.py makemigrations
'''
```

- ``migrate``: Aplica las migraciones a la base de datos.

```
'''bash
```

```
python manage.py migrate
'''
```

Estos comandos permiten mantener la estructura de la base de datos en sincronía con los cambios en los modelos de Django, facilitando el desarrollo y la evolución del proyecto.

---

## Visualización de Base de Datos con DB Browser

DB Browser for SQLite es una herramienta gráfica de código abierto que permite visualizar y gestionar bases de datos SQLite. Esta herramienta es muy útil para verificar la estructura y los datos de la base de datos utilizada en tu proyecto Django.

### Instalación de DB Browser

- 1. Descargar DB Browser:** Visita el sitio web oficial de DB Browser for SQLite y descarga la versión adecuada para tu sistema operativo (Windows, macOS o Linux).
  - 2. Instalar DB Browser:** Sigue las instrucciones de instalación específicas para tu sistema operativo.
- 

## Uso de DB Browser para Verificar la Base de Datos

### Paso 1: Abrir la Base de Datos

- 1. Iniciar DB Browser:** Abre la aplicación DB Browser for SQLite.
- 2. Abrir archivo de base de datos:** Haz clic en "Open Database" y selecciona el archivo de base de datos de tu proyecto Django. Este archivo se encuentra típicamente en el directorio principal del proyecto y se llama ``db.sqlite3``.

### Paso 2: Navegar por las Tablas



- 1. Ver tablas:** Una vez abierta la base de datos, puedes ver las tablas en la pestaña "Database Structure". Aquí encontrarás las tablas creadas por tus modelos de Django.
- 2. Explorar datos:** Selecciona una tabla y haz clic en la pestaña "Browse Data" para ver los registros almacenados en esa tabla.

### Paso 3: Verificar la Estructura de la Tabla

- 1. Estructura de la tabla:** En la pestaña "Database Structure", selecciona una tabla para ver sus columnas y tipos de datos. Esto te permite verificar que la estructura de la tabla coincide con la definida en tus modelos de Django.
- 

## Ejemplo Práctico

Imaginemos que tienes una aplicación Django con un modelo `Estudiante`. Tras ejecutar los comandos `makemigrations` y `migrate`, puedes usar DB Browser para verificar que la tabla `AppCoder_estudiante` se ha creado correctamente.

- 1. Abrir `db.sqlite3` en DB Browser.**
  - 2. Navegar a la tabla `AppCoder_estudiante`.**
  - 3. Verificar columnas como `nombre`, `apellido` y `email`.**
- 

## Conclusión

Las migraciones en Django son una herramienta poderosa para gestionar la evolución de la estructura de la base de datos de manera segura y eficiente. Utilizando comandos como `makemigrations` y `migrate`, puedes mantener tu base de datos en sincronía con tus modelos. Además, herramientas como DB Browser for SQLite facilitan la visualización y verificación de los datos y la estructura de tu base de datos, proporcionando una interfaz gráfica intuitiva para explorar y gestionar tu base de datos SQLite.

---

## Material complementario:

### Recursos complementarios

¿Quieres saber más?

- [Cómo instalar BootStrap 5 en tu proyecto](#) | **Kiko Palomares**